



AWS SUMMIT MÉXICO • SESIÓN DE IA

De la POC a producción

Cómo optimizar, asegurar y escalar agentes de IA en AWS.
Charla y workshop hands-on sobre Amazon Bedrock.



Optimización



Seguridad



Escalamiento



Workshop

3 horas · 75' de charla + 75' de laboratorio

Quiénes somos

Los que vamos a estar con ustedes estas tres horas



Somos una consultora de Cloud y DevOps nacida en Córdoba, Argentina, en 2020. Trabajamos sobre AWS con equipos propios y acompañamos a las empresas en su infraestructura, su plataforma de datos y, desde hace un par de años, sus agentes de IA.

2020

Desde entonces,
sobre AWS

180+

Proyectos entregados
en 12 países

100%

Del equipo técnico
certificado en AWS

- Oficinas en Córdoba y La Plata, y equipo distribuido por LatAm
- Primera empresa de LatAm en el programa EKS Service Delivery de AWS
- Top Partner 2024 de AWS Startups LatAm • Great Place to Work 2023 y 2024



Cómo vamos a trabajar

La primera mitad la miramos juntos; la segunda, cada uno con las manos en el teclado



Todo es reproducible

Cada cosa que mostremos está en el repositorio, con el código y los pasos para correrla en tu cuenta.



Preguntá cuando quieras

No hace falta esperar al final. Si algo no se entiende o querés el detalle de una decisión, cortá y lo vemos.



Te lo llevás funcionando

En la segunda mitad cada uno despliega el agente en su propia cuenta de AWS y se va con él andando.

Si en algún momento se pierden, levanten la mano. Es mejor frenar que seguir sin entender.

Qué vamos a ver

3 horas: 75 minutos de charla y 75 de laboratorio



Bedrock de punta a punta

el mapa completo: qué servicio resuelve qué problema



Optimización

siete estrategias de costo que no son cambiar de modelo



Seguridad

guardrails, prompt injection, identidad del agente y auditoría



Escalamiento

de un MVP a una plataforma multiagente serverless



Workshop

cada uno despliega su propio agente con AWS CDK

01

El mapa completo de Amazon Bedrock

Bedrock no es una API para hablarle a un modelo. Son cuatro familias de servicios, y la mayoría de los equipos usa una sola.

Bedrock en cuatro capas

Todo lo que sigue se agrupa acá

Inferencia y modelos

Converse API • Inference profiles
Batch • Provisioned Throughput
Custom Model Import • Distillation
Marketplace • Fine-tuning

Construcción de agentes

Knowledge Bases (RAG)
Bedrock Agents
AgentCore: Runtime, Gateway,
Memory, Identity, Browser
Flows • Prompt Management

Calidad y datos

Model Evaluation
RAG Evaluation (LLM-as-a-judge)
Bedrock Data Automation
(documentos, imágenes, audio)

Control

Guardrails: contenido, temas,
PII, contextual grounding
Automated Reasoning checks
Model invocation logging

El 80% de los equipos vive en la primera columna. El diferencial está en las otras tres.

Inferencia y modelos

Las decisiones que tomás una vez y pagás todos los meses



Catálogo de modelos

Anthropic, Amazon Nova, Meta, Mistral, Cohere, AI21, Stability. Un mismo contrato de API para todos.



Converse API

Interfaz única, multimodal y con tool use. Cambiar de modelo es cambiar un string, no reescribir el cliente.



Inference profiles

Cross-region: más throughput sin tocar código. Application: etiquetás el consumo y facturás por cliente.



Modos de compra

On-demand para empezar. Batch para lo asíncrono. Provisioned Throughput cuando el volumen es constante.



Modelos propios

Custom Model Import para traer tu modelo ya ajustado. Marketplace para modelos de terceros en tu cuenta.



Distillation y fine-tuning

Potente y sobrevalorado: primero medí cuánto ganás con prompt y contexto.

Construcción de agentes

Tres caminos, y no son excluyentes



Knowledge Bases

RAG gestionado: ingesta, chunking, embeddings y reranking.

Vector stores: OpenSearch Serverless, Aurora pgvector, S3 Vectors, Neptune Analytics.



Bedrock Agents

El agente gestionado de AWS: orquestación, action groups y sesiones sin escribir el loop.

Rápido para casos estándar, menos control fino del prompt.



AgentCore

La pieza nueva y la más ignorada: infraestructura para correr agentes propios en producción.

Sirve con el framework que ya usás.

Regla práctica: Knowledge Bases casi siempre. Agents si el caso es estándar. AgentCore cuando el agente ya es tuyo y el problema es operarlo.

AgentCore, componente por componente

Lo que casi nadie está usando todavía



Runtime

Ejecución serverless de agentes, con sesiones largas y aislamiento por sesión.



Gateway

Expone tus APIs y Lambdas existentes como herramientas MCP, sin reescribir nada.



Memory

Memoria de corto y largo plazo administrada. No es lo mismo que guardar el historial en una tabla.



Identity

Identidad propia del agente y delegación de permisos hacia servicios de terceros.



Browser • Code Interpreter

Herramientas sandboxeadas para navegar y ejecutar código sin exponer tu infraestructura.



Observability

Trazas por sesión: qué herramienta llamó, con qué argumentos y cuánto costó.

Calidad y datos

Sin medición no hay optimización posible



Model Evaluation

Comparás modelos y configuraciones sobre tu propio dataset, con métricas automáticas o jueces humanos.



RAG Evaluation

LLM-as-a-judge sobre el pipeline de recuperación: ¿trajo el documento correcto?, ¿la respuesta se sostiene en la fuente?



Bedrock Data Automation

Convierte documentos, imágenes, audio y video en datos estructurados. El puente entre tu contenido crudo y algo consultable.

La pregunta que ordena el proyecto: ¿Cómo sabés que el cambio que hiciste ayer mejoró el bot y no lo empeoró?

Guardrails: la capa de control

Se configura una vez y aplica a cualquier modelo: podés auditarlo por separado.



Filtros de contenido

Umbral por categoría sobre entrada y salida.



Temas negados

Definís en lenguaje natural de qué no se habla.



Datos personales

Detecta PII y la bloquea o la enmascara.



Palabras y patrones

Listas propias y expresiones regulares.



Contextual grounding

Mide si la respuesta se sostiene en la fuente. El control anti-alucinación.



Automated Reasoning

Validación formal contra reglas declaradas. Para dominios regulados.

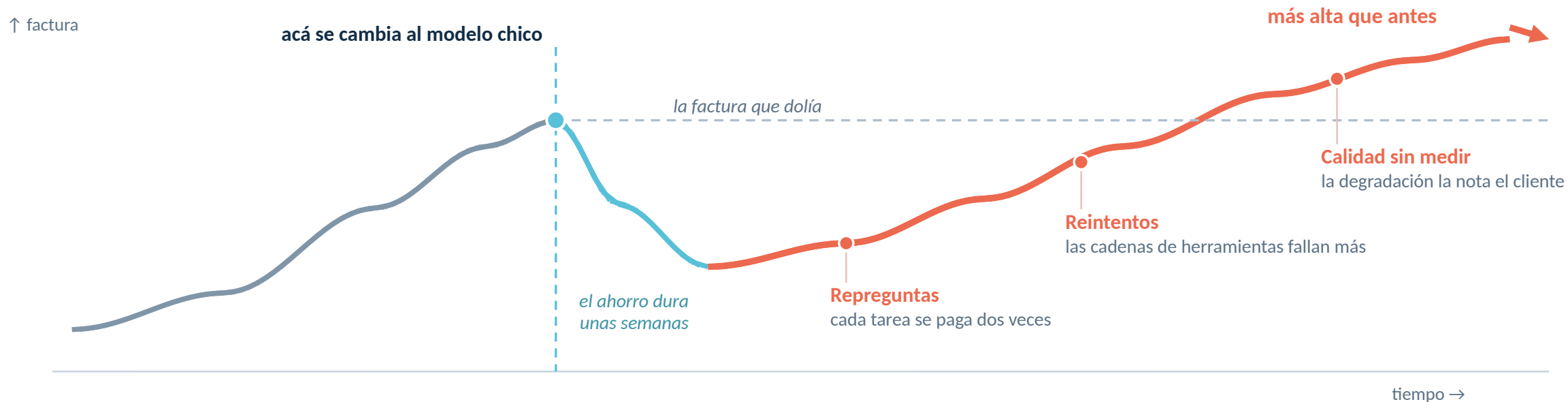
02

Optimización

El costo no se arregla cambiando el modelo. Siete estrategias que sí mueven la aguja, ordenadas por lo que más rinde.

El anti-patrón

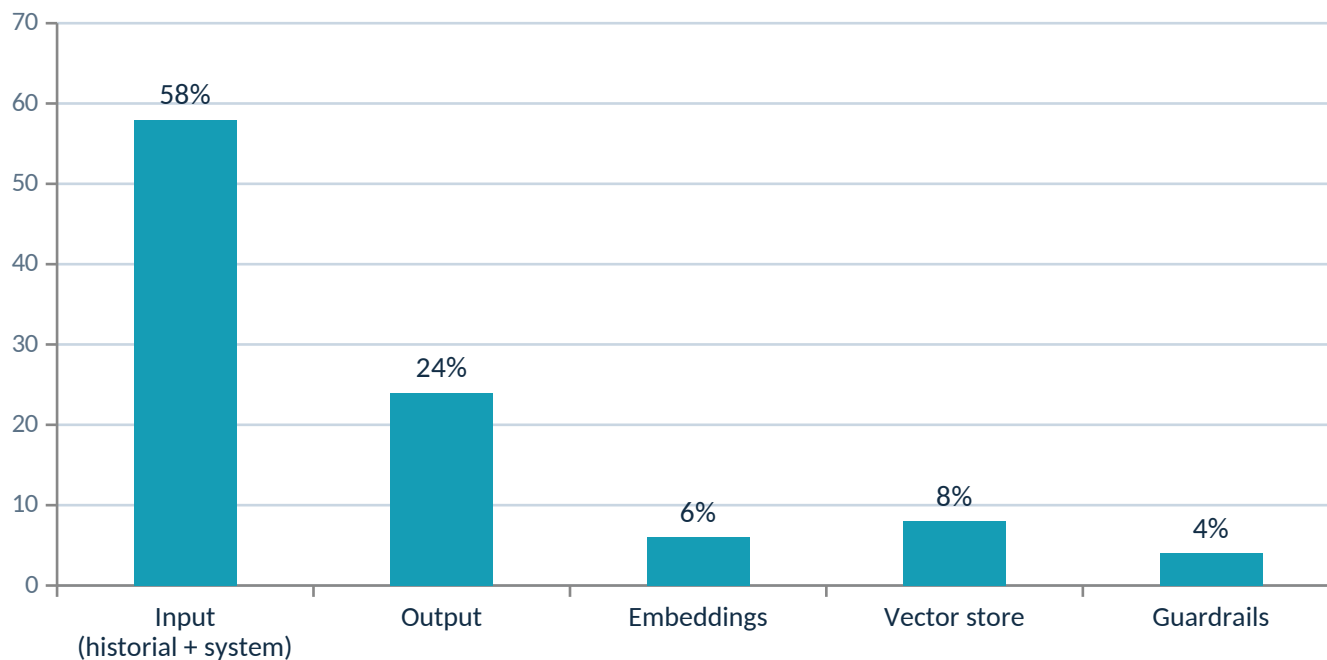
«Me sale caro, me paso a un modelo más chico»



No es que cambiar de modelo esté mal: es que es el paso siete, no el paso uno.

Dónde está el dinero de verdad

Descomposición típica de la factura de un agente conversacional



Proporciones de referencia — reemplazar por la medición del caso propio antes de presentar.

El input es el problema

Lo caro no es lo que escribe el modelo: es lo que le mandás en cada turno, una y otra vez.

Por eso el orden

Caching y contexto atacan la barra más alta. Cambiar de modelo baja el precio, no el volumen.

Routing por tarea

No todas las tareas de un agente necesitan el mismo modelo



Con la **Converse API**, cambiar de modelo es cambiar un string: **modelId**



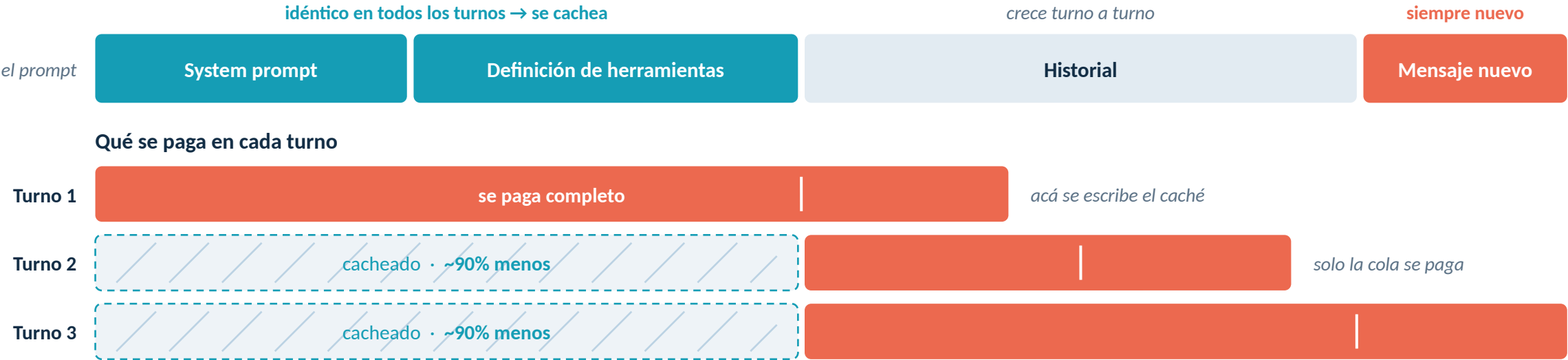
Intelligent Prompt Routing

resuelve parte del ruteo por vos

Los \$ son costo relativo por token: importa el orden de magnitud, no el número exacto

Prompt caching

Lo que no cambia entre turnos no debería pagarse en cada turno



-  Lo estable adelante, lo variable al final
-  Sin timestamps ni IDs en el bloque cacheado
-  Medí el hit rate: si da bajo, el orden está mal

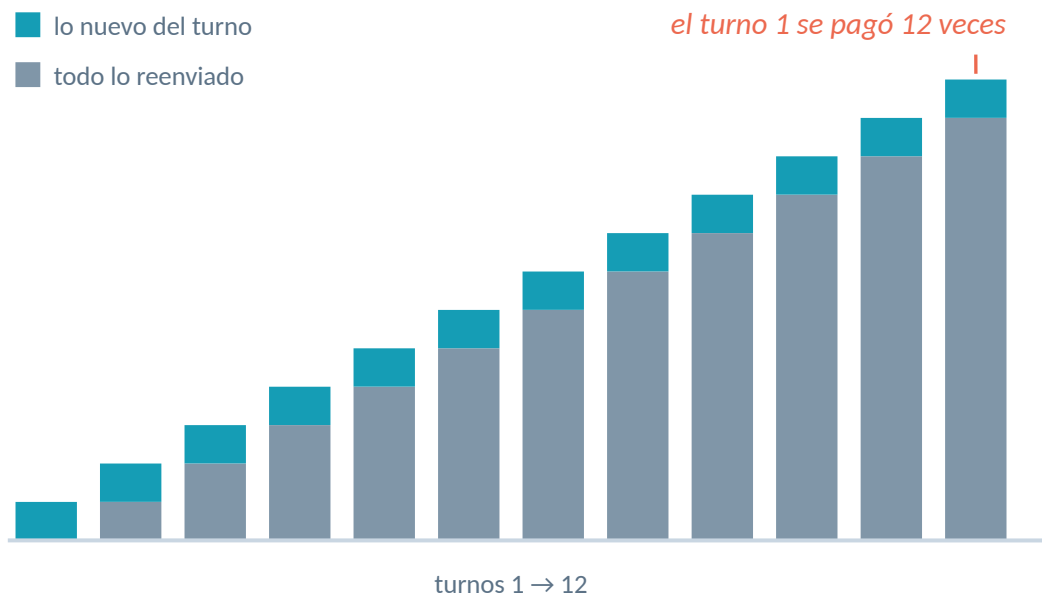
El descuento exacto varía por modelo: ~90% es el orden de magnitud del cache read en Bedrock.

Gestión de contexto

El modelo no recuerda: cada turno reenvía toda la conversación

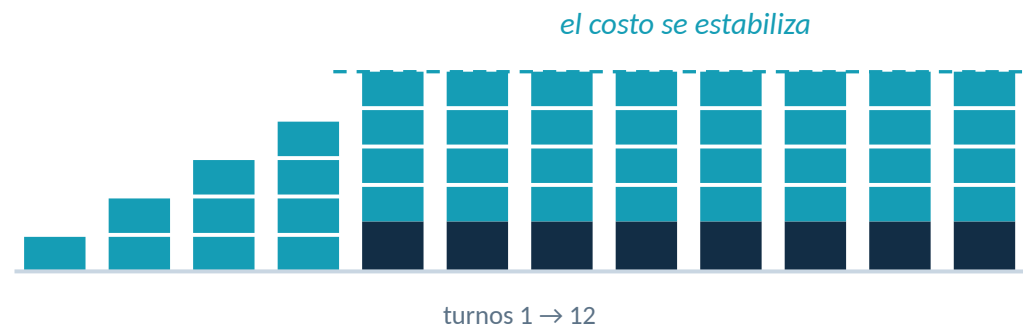
Reenvío completo

- lo nuevo del turno
- todo lo reenviado



Ventana + resumen

- resumen de lo viejo
- últimos 4 turnos completos



Compactación (resumir lo viejo) • **Ventana deslizante** (últimos N completos) • **Resumen incremental** (estado que se actualiza)

Alto de barra = tokens de entrada enviados en ese turno • escala ilustrativa

RAG en lugar de contexto largo

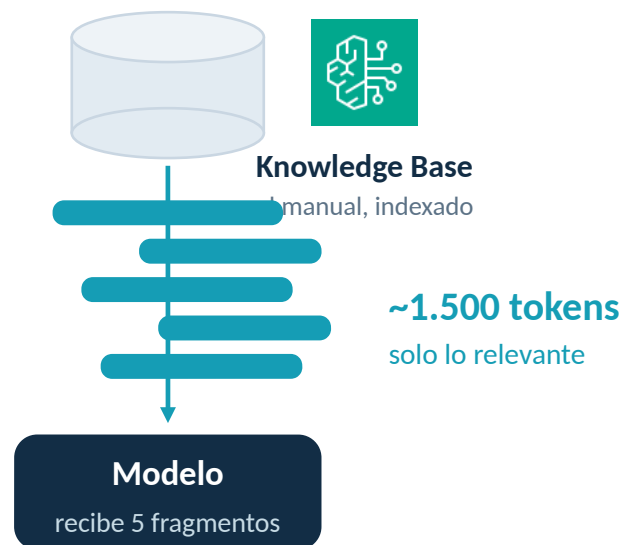
La ventana gigante tienta a pegar el manual entero. Funciona, y lo pagás en cada turno.

El manual entero al prompt



100x
menos tokens por turno

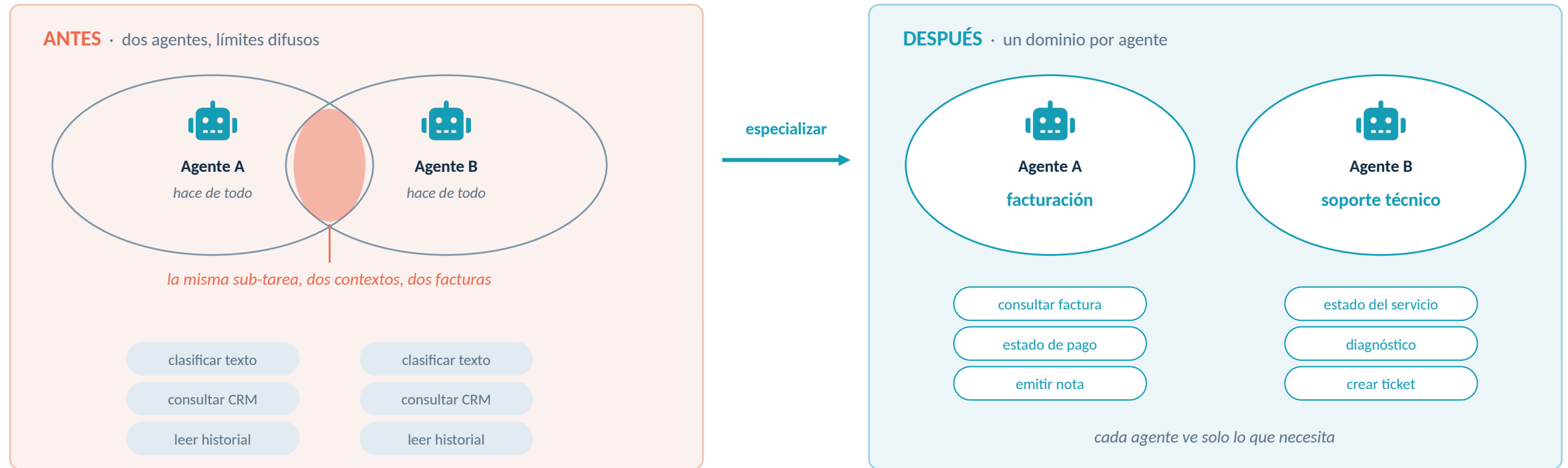
Recuperar lo que hace falta



Menos ruido también es más precisión: el modelo se distrae menos
Knowledge Bases te da ingesta, chunking, embeddings y retrieve sin construirlos

Especialización de agentes

Dos agentes resolviendo lo mismo es pagar lo mismo dos veces

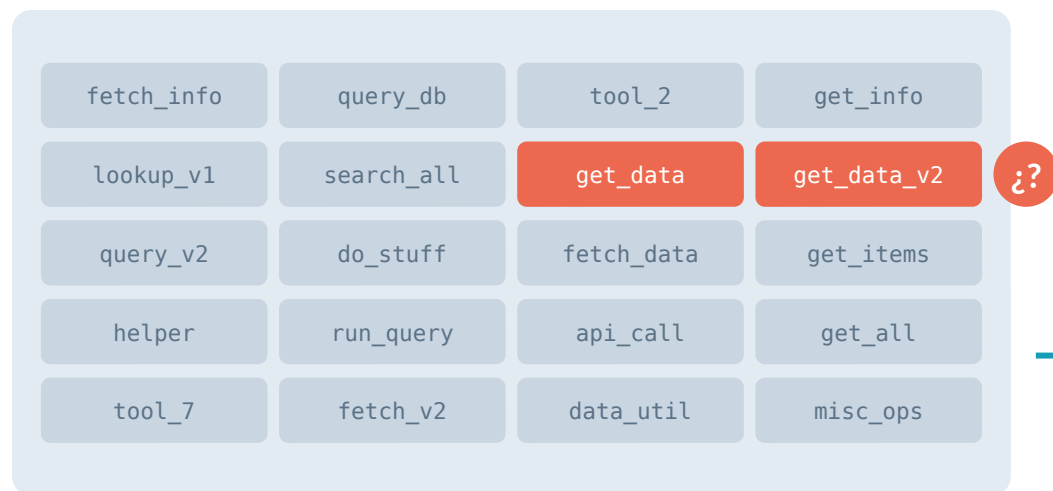


Caso real: dos agentes del mismo sistema clasificaban el mismo texto por separado

Diseño de herramientas

El catálogo de tools viaja completo en cada turno, lo uses o no

20 herramientas mal descritas



~8.000 tokens por turno • y el modelo duda entre parecidas

6 herramientas bien descritas



~1.500 tokens • elección precisa

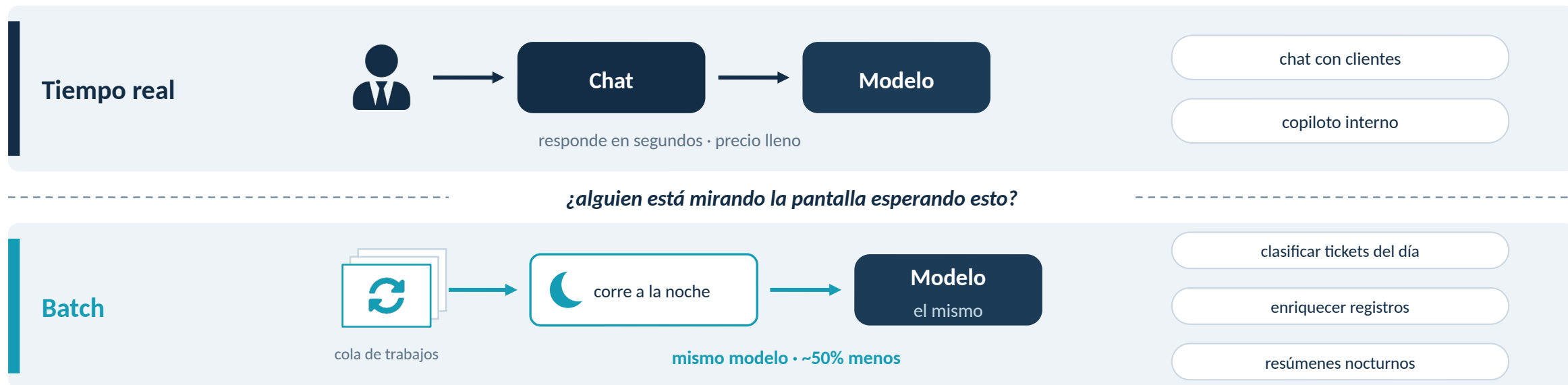
menos y mejor

Más barato en cada turno Y más preciso al elegir

El nombre y la descripción de la tool son prompt: escribilos como prompt

Batch y asincronía

Lo que nadie está esperando no debería pagar precio de tiempo real



Cierre de la serie Ninguna de las siete requiere cambiar de modelo. Aplicadas juntas, el ahorro se acumula sin tocar la calidad.

Batch inference en Amazon Bedrock: ~50% del precio on-demand en los modelos que lo soportan.

Y recién ahora, el modelo

Elegir modelo es una decisión de portafolio, no una bala de plata



El costo escala más rápido

La curva se aplana y el precio sigue subiendo. El modelo grande se paga solo donde el salto de calidad lo justifica.



Medilo sobre tu tarea

La curva cambia por caso de uso: en extracción se aplana antes; en razonamiento, después. Medila con tu dataset.

Valores relativos ilustrativos. La forma de la curva es lo que importa: la tuya se mide, no se hereda.

FinOps para IA

¿Cuánto te cuesta cada cliente que atiende tu bot? ¿Y cada feature?



Application inference profiles

un perfil por cliente, tenant o feature: el consumo queda separado desde el origen



Tags y cost allocation

los perfiles se etiquetan y aparecen discriminados en Cost Explorer



Budgets y alertas

presupuesto por perfil, con alerta antes de que el mes te sorprenda



Métricas de negocio

costo por conversación y por caso resuelto, no solo dólares por millón de tokens



Observabilidad de tokens

logging de invocaciones para saber qué prompt y qué herramienta consumen de más

Sin esto, optimizar es adivinar. Con esto le decís al negocio cuánto cuesta cada conversación.

03

Seguridad

El input es lenguaje natural y la salida no es determinística. Toda tu defensa tradicional asume lo contrario.

Por qué la seguridad tradicional no alcanza

Cambió la superficie de ataque

Aplicación tradicional

El input tiene un esquema validable

La misma entrada da la misma salida

Los permisos son del usuario

El código define qué se puede hacer

Un WAF filtra patrones conocidos

Aplicación con LLM

El input es lenguaje natural: todo es válido

La salida cambia entre ejecuciones

El agente actúa por su cuenta, con permisos propios

El prompt sugiere qué se debería hacer

El ataque viene escrito en español, dentro de un PDF

No reemplaza a tu seguridad actual: se suma una capa nueva, específica del modelo.

Casos reales de la industria

Chatbots que respondieron lo que no debían



El bot que escribía código

Un asistente de atención al cliente respondiendo consultas de programación. Gracioso en Twitter, caro en tokens y pésimo para la marca.



El bot que regalaba descuentos

Usuarios convenciendo al agente de ofrecer condiciones comerciales que no existían, y después reclamando que se cumplieran.



El bot que filtró contexto

Instrucciones internas y datos de otros usuarios apareciendo en la respuesta porque todo vivía en el mismo prompt.

Qué falló en los tres casos

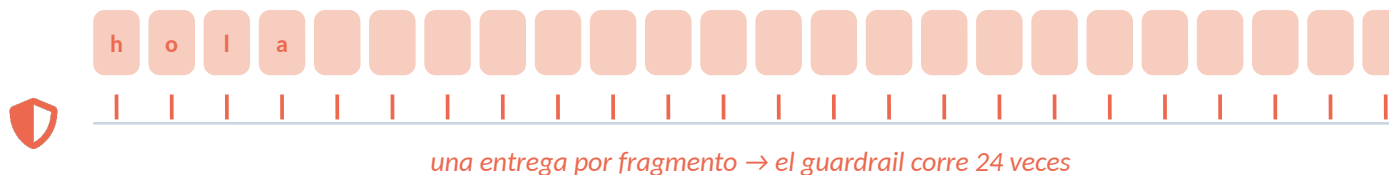
No fue el modelo: fue que nadie definió el perímetro. Ningún tema negado, ninguna validación de la salida, ningún aislamiento entre lo que el usuario escribe y lo que el sistema instruye.

El costo oculto de los guardrails

Por qué la industria está abandonando el streaming carácter por carácter

Streaming fino

carácter por carácter



24

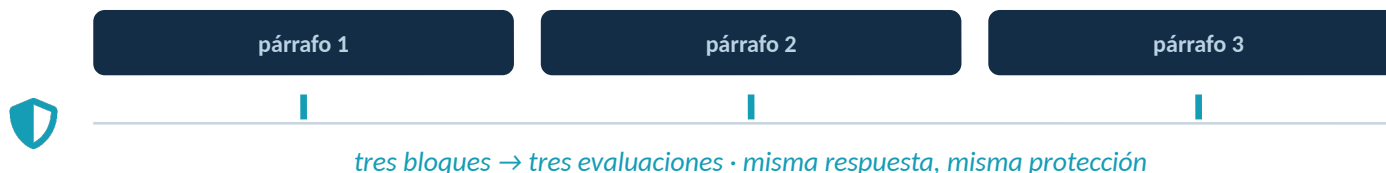
evaluaciones por respuesta

El guardrail es un middleware sobre la salida: corre una vez por cada entrega — la granularidad la elegís vos

Entrega por bloques

como responden los agentes hoy:

bloques, tarjetas, resultados



3

evaluaciones por respuesta

No es estética: es arquitectura. El tamaño del bloque que entregás define cuánto cuesta y cuánta latencia agrega tu capa de seguridad.

Sin guardrail en la salida, el argumento no aplica: ahí el streaming fino es solo una decisión de experiencia.

Prompt injection

La mitigación real no es un prompt más firme: es que el agente no tenga permisos para hacer el daño.

Directa

El usuario escribe «ignora tus instrucciones». La más fácil de mitigar.

Indirecta

Viene dentro de un documento, un ticket o una web que el agente lee.



Separá datos de instrucciones

lo recuperado es información para razonar, nunca una orden a ejecutar



Curá la fuente

quién sube documentos al knowledge base es una decisión de seguridad, no de producto



Permisos mínimos por herramienta

si el agente no puede transferir dinero, ninguna instrucción inyectada lo logra



Confirmación humana

toda acción irreversible pasa por una persona, siempre

Identidad, perímetro y auditoría

Lo que hace auditable a un agente



Identidad del agente

Rol propio y permisos mínimos. Nunca usa las credenciales del usuario.



Perímetro de red

VPC endpoints para Bedrock, WAF adelante, Cognito y cifrado con KMS.



Auditoría

Model invocation logging, CloudTrail y trazas por sesión.



Aislamiento entre usuarios

Sesión, memoria y recuperación filtradas por tenant, en la plataforma.



Secretos

Nada de claves en el prompt ni en el código: Secrets Manager y rotación.



Retención

Cuánto guardás de cada conversación y por cuánto. Es decisión legal.

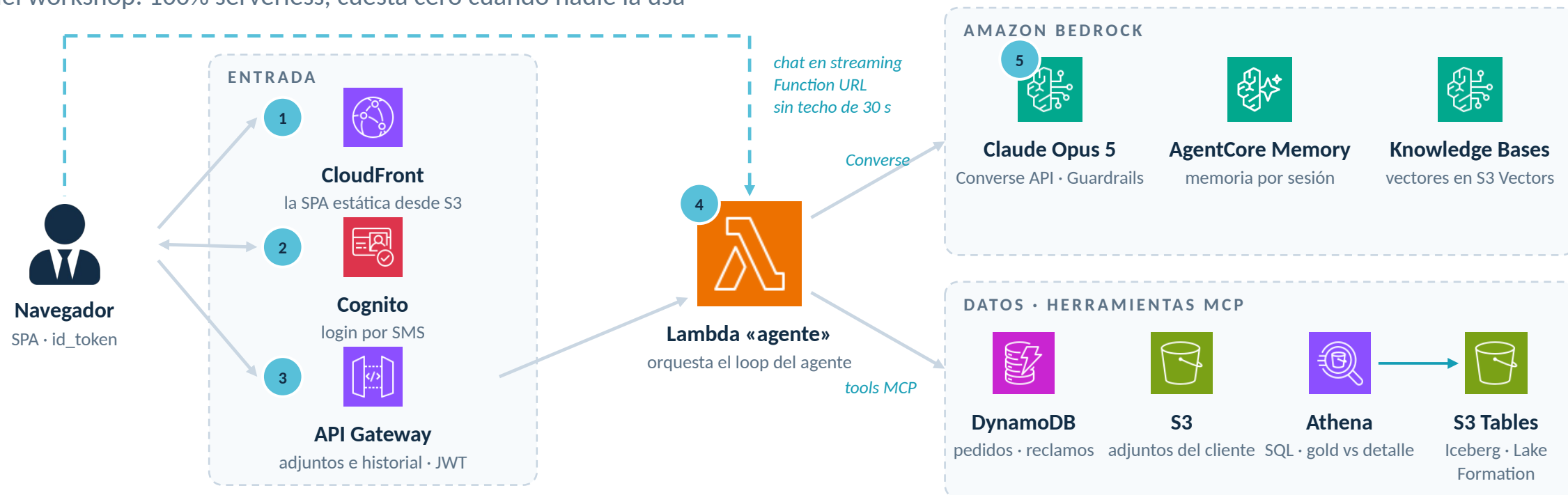
04

Escalamiento

El MVP anda. La pregunta es qué pasa cuando entran diez mil usuarios y el equipo que lo mantiene son dos personas.

Arquitectura de referencia

La del workshop: 100% serverless, cuesta cero cuando nadie la usa



Sin servidores que mantener y con cada pieza escalando por separado. Es exactamente lo que despliega el workshop.

El chat entra por Function URL — streaming sin el techo de 30 s de API Gateway; adjuntos e historial van por API Gateway con autorizador JWT.

El mismo agente, gestionado

Cada pieza que escribimos a mano existe como servicio de AgentCore

A mano — lo que desplegamos hoy

El loop del agente

~100 líneas en la Lambda: prompt, tools, reintentos, cierre

El catálogo de tools

registro.ts: qué herramientas existen y quién las ejecuta

La memoria

AgentCore Memory — ya era el servicio gestionado

La identidad

el clientId sale del JWT; las tools no le creen al modelo

Gestionado — Bedrock AgentCore

AgentCore Harness

modelo + prompt + tools + memoria como recurso CFN

AgentCore Gateway

las mismas Lambdas, expuestas como servidor MCP central

AgentCore Memory

idéntica: por actor y sesión, expira sola

Identity · Policy Engine

interceptores y reglas Cedar por cada tool call

Lo que no cambia: las herramientas Lambda, DynamoDB, la Knowledge Base y el lakehouse son los mismos. Solo se reemplaza el andamiaje.

En el repo está dos veces: workshop/ con el loop a mano para entenderlo, y workshop-full-bedrock/ con la suite gestionada.

Harness, Gateway y Memory se declaran por CloudFormation; el chat de la versión gestionada corre por streaming con InvokeHarness.

De un agente a muchos

Y cuándo no hacerlo



Supervisor y subagentes

Un agente coordina y delega en especialistas. Cada uno con su contexto y sus herramientas: menos tokens y menos confusión.



Agente como tool

Un agente expuesto como herramienta de otro. Más simple de razonar que un enjambre conversando entre sí.



MCP como contrato

Sumar una capacidad es sumar un server MCP, no reescribir el agente. AgentCore Gateway expone lo que ya tenés.



Cuándo NO

Si un agente con buenas herramientas resuelve el caso, multiagente solo agrega latencia, costo y superficie de error.

La pregunta no es «cuántos agentes», es «cuántos contextos distintos necesito mantener separados».

Los techos con los que te vas a chocar

Ninguno aparece en el demo

Cuotas de Bedrock	Tokens y requests por minuto por modelo y región.	Cross-region inference profiles y aumento de cuota con tiempo.
Throttling	Los picos que superan la cuota devuelven error en la demo del cliente.	Reintentos con backoff, colas y degradación a otro modelo.
Concurrencia de Lambda	El límite de la cuenta se comparte con todo lo demás.	Concurrencia reservada, y provisionada si molesta el arranque frío.
Timeouts	Un agente con varias herramientas tarda más que un request HTTP típico.	Procesamiento asíncronico y respuesta por eventos.
Costo del crecimiento	El gasto crece con los usuarios y nadie lo proyectó.	Presupuesto por perfil de inferencia y alertas antes del cierre.

05

Workshop

75 minutos. Cada uno despliega un agente de atención al cliente en su propia cuenta de AWS, con CDK.

Qué vamos a construir

Un agente de atención al cliente, 100% serverless



Login por SMS

Cognito con código de un solo uso al celular.
Sin contraseñas.



Chat con Claude

Claude Opus 5 en Bedrock, vía Converse API
con tool use.



Imágenes

El cliente manda una foto del producto y el
agente la interpreta.



Knowledge Base (MCP)

Responde con la base de conocimiento de la
empresa.



DynamoDB (MCP)

Consulta pedidos, estado e historial del
cliente.



Reclamos

Crea el ticket y devuelve el ID. Guardrails
activos sobre todo el flujo.

Serverless de punta a punta

Costo fijo mensual ≈ 0 : pagás cuando alguien conversa, no por tener el sistema prendido.

Servicio	Qué resuelve	En reposo
 CloudFront + S3	El chat servido global, sin servidores web	pago por request
 Cognito	Login por SMS, sin contraseñas que proteger	por usuario activo
 API Gateway	Autorizador JWT delante de la Lambda	pago por request
 Lambda	El orquestador del agente y sus herramientas	\$0 sin invocaciones
 Bedrock	Claude Opus 5, Guardrails y embeddings	pago por token
 DynamoDB	Pedidos, sesión, tickets y vectores	\$0 sin requests
 S3	Fotos del cliente, con expiración a 7 días	por GB-mes

El loop del agente, completo

Todo lo que vimos, en una pantalla de código

```
lambda/agent/bedrock.ts

const respuesta = await cliente.send(
  new ConverseCommand({
    modelId: "us.anthropic.claude-opus-5",
    messages,
    system: [{ text: PROMPT_SISTEMA }],
    toolConfig: toolConfigBedrock(),
    guardrailConfig: { guardrailIdentifier, guardrailVersion },
  }),
);
// pidio herramientas: se ejecutan y vuelven al modelo
if (respuesta.stopReason === "tool_use") {
  const salida = await ejecutarHerramientas(respuesta);
  messages.push({ role: "user", content: salida });
}
```



Un solo contrato

Converse habla igual con Claude, Nova o Llama.



El guardrail es config

Se declara en la infraestructura, no en el código.



El loop es corto

Pide, ejecutás, devolvés. Con tope de vueltas.

Esto es todo el agente. El resto de la complejidad está en las herramientas y en los permisos.

El laboratorio, paso a paso

Seis pasos · 75 minutos

1

Setup

cuenta, credenciales, bootstrap de CDK y clone del repositorio — 10 min

2

La arquitectura

recorremos el código: qué hace cada stack y por qué está así — 15 min

3

Deploy

cdk deploy y la infraestructura queda arriba en tu cuenta — 15 min

4

Knowledge Base

cargamos los documentos y probamos el primer prompt end-to-end — 15 min

5

Tu propia herramienta MCP

el ejercicio que se llevan: sumar una tool nueva al agente — 15 min

6

Romperlo

probamos los guardrails, miramos las trazas y el costo de la sesión — 5 min

Si algo falla en tu cuenta, hay un entorno ya desplegado por nosotros para que sigas el ejercicio igual.

Antes de empezar

Chequeo rápido: levánta la mano si te falta alguno



Cuenta AWS

Con acceso a Amazon Bedrock y los modelos de Anthropic habilitados en la región del laboratorio.



Node 20 o superior

Y el AWS CLI configurado con credenciales que puedan crear recursos.



Un celular

Para recibir el código de acceso por SMS del login de Cognito.



github.com/craftech-io/aws-summit-mx-2026

El código completo, el README con los pasos, los datos de la Knowledge Base y estas presentaciones. Clonalo ahora así arrancamos juntos.

Lo que te llevás

Cinco cosas concretas



El mapa de Bedrock

para decidir arquitectura el lunes, no solo InvokeModel



Siete estrategias de costo

aplicables sin cambiar de modelo y sin degradar la calidad



Un checklist de seguridad

guardrails, injection, identidad del agente y auditoría



Los techos de escala

cuotas, throttling y concurrencia, con su mitigación



Un repo funcionando

desplegado en tu cuenta, para extender con tus propias herramientas

Gracias

- El costo no se arregla cambiando el modelo: se arregla con arquitectura.
- La seguridad en IA es una capa propia, no la que ya tenías.
- Escalar es un problema de cuotas, contexto y observabilidad.
- Todo lo que vimos está en el repo, listo para desplegar.

Autonomía

Pasión

Confianza

Trabajo en Equipo

Innovación

Compromiso